IAM-03: Group Architecture and Design

> **Series:** IAM — IAM Administration | **Notebook:** 3 of 12 | **Created:** January 2026 | **Last Updated:** 10/13/2026

Designing Scalable Group Structures

Groups are the foundation of access management. A well-designed group architecture simplifies administration, improves security, and scales with your organization.

Table of Contents

1. Group Fundamentals
2. Group Design Patterns
3. Naming Conventions
4. SAML Group Mapping
5. Group Lifecycle Management
6. Separation of Duties
7. Cross-Environment Patterns
8. Group Assessment Queries

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled

Requirement	Details
Permissions	`account-user-management` or group management rights
Prior Knowledge	**IAM-01** and **IAM-02** (SSO configured)

1. Group Fundamentals

Groups in Dynatrace Gen3 IAM are collections of users that share access permissions. Groups exist at the **account level** and can be assigned to one or more environments.

Group Properties

Property	Description	Required
Group name	Unique identifier for the group	Yes
Description	Purpose and ownership information (max 255 characters) — groups have no separate owner field, so record the owner here	Recommended
Security claims (SAML Group Attribute Value)	Federated group values that map IdP users into the group	Optional — only with an active SAML federation

What Groups Control

Groups are assigned:
- **Account-level policies** (apply to all environments)
- **Environment-level policies** (scoped to specific environments)
- **Boundaries** (attached to each policy binding, narrowing what that policy grants)

Permissions Come Only Through Groups

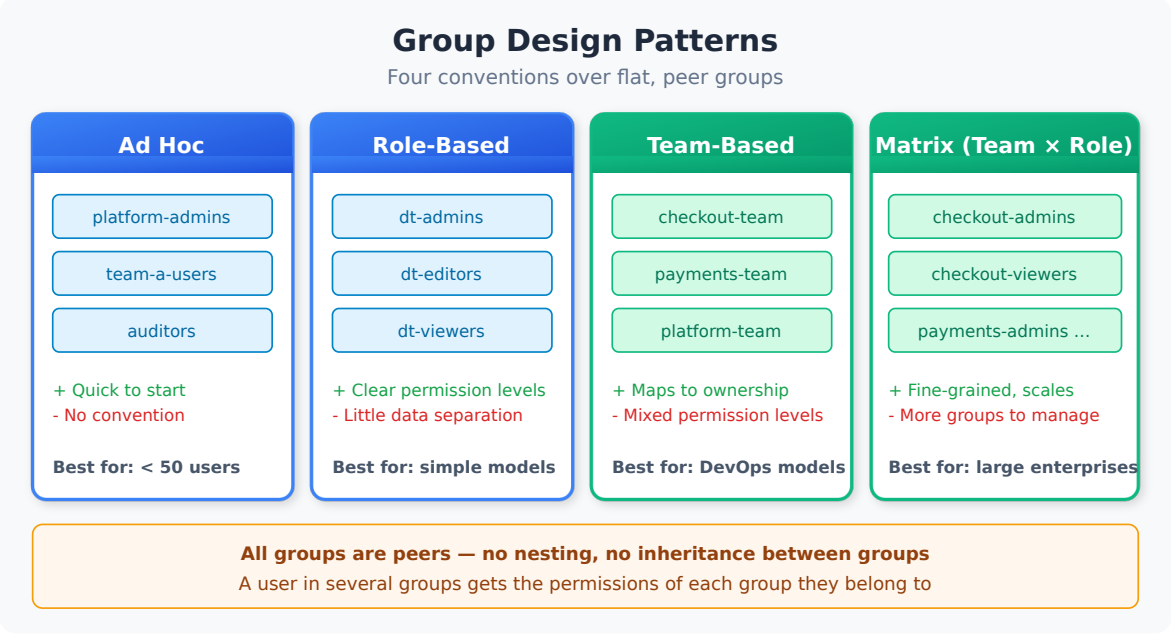
Users get their Dynatrace permissions from the groups they belong to — there is no direct policy assignment to a user. Temporary or break-glass access is therefore a **group membership** change (see the break-glass group in Section 6), which keeps every grant visible in group membership and the account audit log.

> **Sources:** Group management (DT docs) — *"users inherit the Dynatrace access permissions that are assigned to the Dynatrace groups they belong to."*

2. Group Design Patterns

Choose a structure that matches your organization and scales appropriately. Dynatrace groups are **flat** — the group documentation describes no nesting and no inheritance between groups — so every pattern below is a naming and membership convention over peer groups. A user who belongs to several groups gets the permissions of each.

> **Sources:** Group management (DT docs) — *"users inherit the Dynatrace access permissions that are assigned to the Dynatrace groups they belong to."*



Pattern 1: Ad Hoc

Groups created as needs arise, with no role or team convention.

```
Groups:
├── platform-admins
├── platform-viewers
├── team-a-users
├── team-b-users
└── auditors
```

Pros: Simple, easy to understand

Cons: Doesn't scale — no role or team convention to keep group names and grants consistent

Pattern 2: Role-Based

Groups organized by function/role.

```
Groups:
├── dt-admins              (full access)
├── dt-editors             (read + write)
├── dt-viewers             (read only)
└── dt-auditors            (compliance access)
```

Pros: Clear permission levels

Cons: Limited data segregation

Pattern 3: Team-Based

Groups organized by team ownership.

```
Groups:
├── checkout-team          (checkout service owners)
├── payments-team          (payments service owners)
├── platform-team          (infrastructure owners)
└── sre-oncall             (incident response)
```

Pros: Maps to ownership model

Cons: Permission levels mixed within groups

Pattern 4: Matrix (Recommended for Enterprise)

Combines role and team dimensions.

```
Groups:
├── checkout-admins        (checkout team, admin access)
├── checkout-editors       (checkout team, edit access)
├── checkout-viewers       (checkout team, view access)
├── payments-admins
├── payments-editors
├── payments-viewers
├── platform-admins        (cross-cutting admin)
└── all-viewers            (everyone, read-only)
```

Pros: Fine-grained control, scales well
Cons: More groups to manage

Choosing a Pattern

Organization Size	Recommended Pattern
< 50 users	Ad hoc or Role-Based
50-500 users	Team-Based or Matrix
> 500 users	Matrix

3. Naming Conventions

Consistent naming makes groups discoverable and self-documenting.

Recommended Format

```
&lt;scope&gt;-&lt;team/domain&gt;-&lt;role&gt;
```

Examples:

Group Name	Scope	Team	Role
`dt-checkout-admins`	Dynatrace	Checkout	Admin
`dt-payments-viewers`	Dynatrace	Payments	Viewer
`dt-platform-editors`	Dynatrace	Platform	Editor

Group Name	Scope	Team	Role
`dt-all-oncall`	Dynatrace	All	On-call access

Naming Rules

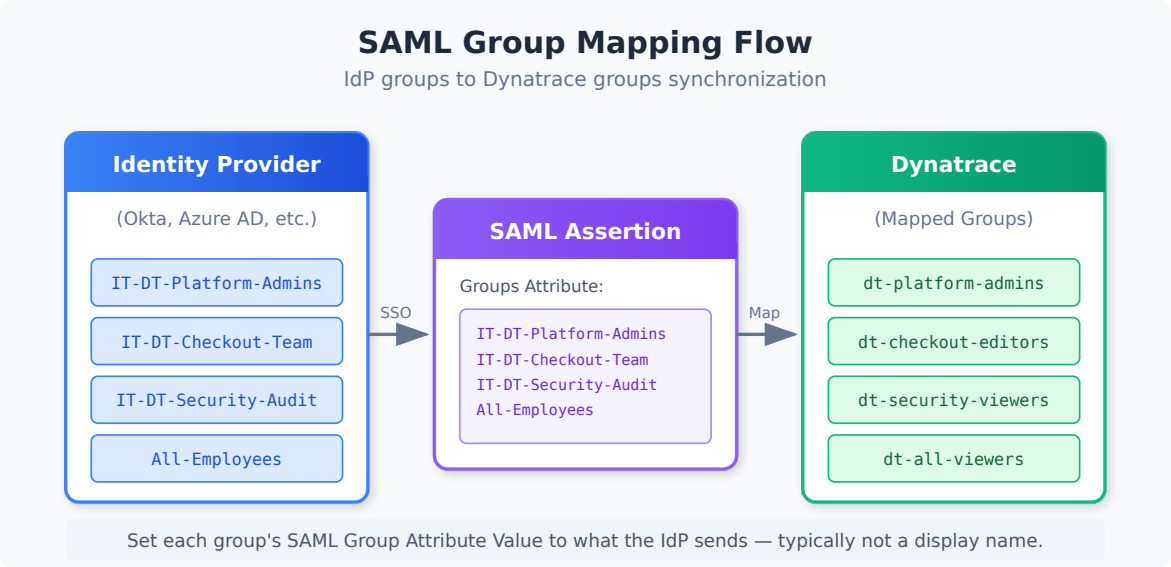
Rule	Rationale
Use lowercase	Consistency, SAML compatibility
Use hyphens (not underscores)	URL-safe, readable
Prefix with `dt-`	Identify Dynatrace groups in IdP
Keep under 50 characters	Display limits
Avoid abbreviations	Clarity over brevity

Anti-Patterns to Avoid

Bad Name	Problem	Better Name
`Team A`	Spaces, generic	`dt-team-a-users`
`ADMINS`	Uppercase, no context	`dt-platform-admins`
`grp_dt_chk_adm`	Cryptic abbreviations	`dt-checkout-admins`
`dynatrace-users`	Too generic	`dt-all-viewers`

4. SAML Group Mapping

Federating groups from your Identity Provider (IdP) automates user provisioning and ensures consistent access.



How It Works

1. User authenticates via SAML SSO
2. IdP includes group memberships in SAML assertion
3. Dynatrace adds the user to every Dynatrace group whose SAML Group Attribute Value matches a value in the assertion
4. User receives permissions from mapped groups

Mapping Strategies

Strategy	Description	Use When
1:1 Mapping	IdP group = DT group	Simple, clear ownership
Many:1 Mapping	Multiple IdP groups → one DT group	Consolidating access
1:Many Mapping	One IdP group → multiple DT groups	Granting multiple roles

IdP Group Attribute

Configure your IdP to send groups in the SAML assertion:

Azure AD (Entra ID):

```
Claim name: groups
Source attribute: user.assignedgroups
```

Okta:

```
Attribute: groups
Value: Matches regex: dt-.*
```

OneLogin:

```
SAML Attribute: groups
Value: Member Of (filtered)
```

Mapping Configuration in Dynatrace

Mapping is set **on each Dynatrace group**: Account Management → **Identity & access management** → **Group management** → the group's **Edit** pane → **SAML Group Attribute Value**. Enter the value(s) your IdP sends in the groups attribute — typically an identifier such as an LDAP ID, not the group's display name. One group can list several values (many-to-one), and the same value can be entered on several groups (one-to-many).

Example Mappings:

IdP Group	Dynatrace Group
`IT-Dynatrace-Admins`	`dt-platform-admins`
`App-Checkout-Team`	`dt-checkout-editors`
`All-Employees`	`dt-all-viewers`

> ⚠️ **Converting a LOCAL group to SAML removes all its members.** Adding a SAML Group Attribute Value to an existing local group switches it to SAML and drops every current user assignment — plan the cutover per group and confirm the IdP sends the value first.
>
> **Sources:** SAML (DT docs) — *"This typically isn't a group display name."*, *"Switching a LOCAL group to SAML removes all user assignments to that group."*

5. Group Lifecycle Management

Groups have a lifecycle that requires ongoing management.

Group Lifecycle Stages

Stage	Actions	Owner
Request	Business justification, approval	Requestor
Create	Create group, configure policies	IAM Admin
Populate	Add initial members	IAM Admin / Group Owner
Operate	Ongoing membership changes	Group Owner
Review	Periodic membership audit	Group Owner + Auditor
Retire	Remove members, delete group	IAM Admin

Group Owner Responsibilities

Every group should have a designated owner who:

- Approves membership requests
- Removes departed employees
- Participates in access reviews
- Maintains group description/documentation

Membership Change Process

Adding Members:
1. User or manager requests access
2. Group owner approves request
3. IAM admin (or automated process) adds user
4. User confirms access works

Removing Members:
1. Triggered by: role change, termination, access review
2. Group owner authorizes removal

3. IAM admin removes user
4. Audit log captures change

Periodic Access Reviews

Group Type	Review Frequency	Reviewer
Account admin groups	Monthly	CISO / Security
Admin groups	Quarterly	Security + Account Admin
Editor groups	Semi-annually	Group Owner + Manager
Viewer groups	Annually	Group Owner

See **IAM-07: Audit Logging and Compliance** for access review automation.

6. Separation of Duties

> **No read-only IAM role yet.** Dynatrace briefly announced a read-only `View users and groups` role in the SaaS 1.346 release notes; the page's changelog records that entry as removed on 09/01/2026. Until such a role ships, give inspection-only staff `account-viewer`, and route IAM change review through the account audit log (IAM-07).
>
> **Sources:** What's new in Dynatrace SaaS 1.346 (DT docs).

Design your group structure to enforce separation of duties and reduce risk.

Key Separations

Separation	Groups to Separate	Reason
Admin vs User	`dt-platform-admins` vs `dt-all-viewers`	Prevent privilege escalation
Dev vs Prod	`dt-app-dev-*` vs `dt-app-prod-*`	Protect production
Configure vs Operate	`dt-config-editors` vs `dt-operations`	Change control

Separation	Groups to Separate	Reason
Audit vs Admin	`dt-auditors` vs `dt-*-admins`	Independent oversight

Incompatible Group Pairs

Users should NOT be in both groups simultaneously:

Group A	Group B	Risk
Production Admins	Development Editors	Dev-to-prod escalation
Security Auditors	Platform Admins	Self-audit
Token Managers	Application Users	Token abuse

Enforcement Options

1. **Manual Review**: Check during access reviews
2. **IdP Rules**: Configure mutually exclusive groups in IdP
3. **Automation**: Script to detect violations

Break-Glass Groups

Emergency access should be handled through special groups:

```
dt-breakglass-admins
├── Empty by default
├── Temporary membership only (&lt; 24h)
├── Requires documented incident
└── Automatic expiration
```

7. Cross-Environment Patterns

When you have multiple Dynatrace environments, design groups that work across them.

Environment Types

Environment	Purpose	Typical Access
Production	Live customer data	Restricted, audit-heavy
Non-Production	Dev, test, staging	Broader access
Sandbox	Learning, experimentation	Open access

Cross-Environment Group Patterns

Pattern A: Environment-Specific Groups

```
dt-prod-checkout-admins    (prod env only)
dt-nonprod-checkout-admins (nonprod env only)
```

Pattern B: Role Groups + Environment Assignment

```
dt-checkout-admins
├── Assigned to Prod env with boundary X
└── Assigned to NonProd env with boundary Y
```

Pattern C: Tiered Groups

```
dt-tier1-admins  (all environments, full access)
dt-tier2-admins  (nonprod only, full access)
dt-tier3-viewers (all environments, view only)
```

Recommendations

Scenario	Recommended Pattern
Strong prod/nonprod separation	Pattern A (env-specific)
Simplified management	Pattern B (role + assignment)
Clear access tiers	Pattern C (tiered)

8. Group Assessment Queries

Use these queries to understand your current group usage.

> **Group, policy, boundary and binding changes are account-level, and this environment's audit events are not their change log.** *"Dynatrace provides audit logs of all changes to your account-level identity and access (IAM) management settings" — "Administrators can view these logs in Account Management > Settings > Audit log"*, where they are kept for up to ten years (also queryable through the Account Audits API). The query below shows IAM API calls made **through this environment** — useful for spotting automation, not a change log. On the validation tenant, 30 days of these events held no group or policy change at all: only user lookups and scheduled `/lookups/iam_*` file uploads, which the query now excludes.
>
> **Sources:** Account Management audit logs (DT docs).

```
// IAM API activity in this environment — recent calls
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

```
// IAM API activity in this environment over 90 days (group creation itself is
account-level)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

```
// IAM API activity in this environment, by user
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| summarize changes = count(), by:{user.id, event.type}
| sort changes desc
| limit 25
```

Next Steps

With your group architecture defined, proceed to configure access controls:

Recommended Path

1. **IAM-04: Policy Authoring and Management** - Create policies for your groups
2. **IAM-05: Boundary Design Patterns** - Define what each group can see
3. **IAM-06: User Lifecycle and Provisioning** - Automate user management

> **Workshop:** See **IAM-11: Policy Persona Workshop** for a guided exercise on mapping personas to groups.

Group Architecture Checklist

Before moving on, ensure you have:

- [ ] Chosen a group design pattern
- [ ] Defined naming conventions
- [ ] Configured SAML group mappings (if using SSO)
- [ ] Documented group ownership
- [ ] Planned separation of duties
- [ ] Designed cross-environment patterns (if applicable)

Summary

In this notebook, you learned:

- Group fundamentals and properties
- Four flat group design patterns: ad hoc, role-based, team-based, and matrix
- Naming conventions for scalable group management
- SAML group mapping strategies
- Group lifecycle management and ownership
- Separation of duties through group design
- Cross-environment group patterns

References

- User Groups
- SAML 2.0 Integration
- Manage User Permissions

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.